Fundamentos de Bases de Datos

-Facultad de ciencias, UNAM

M.I. Gerardo Avilés Rosasgar@ciencias.unam.mx

Tarea 4 **Álgebra Relacional**

Fecha de entrega **30 de octubre de 2026.**

1. Cardinalidad de la consulta

Considera las siguientes relaciones:

R =

A	B
1	x
2	y
2	z
3	x
9	a

S =

B	C	D
x	0	3
y	2	1
y	3	3
w	3	0
y	4	2

Para las siguientes expresiones de **álgebra relacional**, completa la tabla con el número de tuplas que cada una de ellas produce utilizando las relaciones **R** y **S**. Deberás indicar las **tablas resultantes** en cada caso

Expresión	Cardinalidad del resultado
R x S	25
R ⋈ D > A S	7
R ⟖ S	7
R ⟕ S	6
R ⋈ A = D S	5
ρ *C* □ *A (R)* ⋈ *S*	1
π *B (R)* – π *B (σ C ≥ 3 (S))*	3
π *A (R)* ∩ ρ *A* □ *D* (π *D (S))*	3
π *D (S)* ⋈ *R*	20

γ *A; count(B)* □ *t (R ⋈ S)*	5

1. $R \times S = \{tq \mid t \in R \wedge q \in S\}$

	A	B	B	C	D
1	1	x	x	0	3
2	1	x	y	2	1
3	1	x	y	3	3
4	1	x	w	3	0
5	1	x	y	4	2
6	2	y	x	0	3
7	2	y	y	2	1
8	2	y	y	3	3
9	2	y	w	3	0
10	2	y	y	4	2
11	2	z	x	0	3
12	2	z	y	2	1
13	2	z	y	3	3
14	2	z	w	3	0
15	2	z	y	4	2
16	3	x	x	0	3
17	3	x	y	2	1
18	3	x	y	3	3
19	3	x	w	3	0
20	3	x	y	4	2
21	9	a	x	0	3
22	9	a	y	2	1
23	9	a	y	3	3
24	9	a	w	3	0
25	9	a	y	4	2

2. $R \bowtie_{D>A} S$

D < A

	A	B	B	C	D
1	1	x	x	0	3
2	1	x	y	3	3
3	1	x	y	4	2
4	2	y	x	0	3
5	2	y	y	3	3
6	2	z	x	0	3
7	2	z	y	3	3

3. $R \bowtie S$

	A	B	C	D
1	1	x	0	3
2	2	y	2	1
3	2	y	3	3
4	2	y	4	2
5	2	z	null	null
6	3	x	0	3
7	9	a	null	null

4. $R \bowtie_{=} S$

	A	B	C	D
1	1	x	0	3
2	3	x	0	3
3	2	y	2	1
4	2	y	3	3
5	null	w	3	3
6	2	y	4	2

5. $R \bowtie_{A=D} S$

	A	B	B	C	D
1	1	x	y	2	1
2	2	y	y	4	2
3	2	z	y	4	2
4	3	x	x	0	3
5	3	x	y	3	3

6. $\rho_{C \leftarrow A}(R) \bowtie S$

	C	B	D
1.	2	y	1

	~~A~~ C	B	B	C	D
1.	2	y	y	2	1

7. $\pi_{B}(R) - \pi_{B}(\sigma_{C \geq 3}(S))$

	B
1	x
2	z
3	a

R S

B	B	C
x	x	0
y	y	2
z	y	3
x	w	3
a	y	4

≥3

8. $\pi_{A}(R) \cap \rho_{A \leftarrow D}(\pi_{D}(S))$

	A
1	1
2	2
3	3

R S

A	~~D~~ A
1	3
2	1
~~1~~	~~3~~
3	0
9	2

9. $\Pi_{D(S)} \bowtie R$

S

	D	A	B
1	3	1	x
2	3	2	y
3	3	2	z
4	3	3	x
5	3	9	a
6	1	1	x
7	1	2	y
8	1	2	z
9	1	3	x
10	1	9	a
11	0	1	x
12	0	2	y
13	0	2	z
14	0	3	x
15	0	9	a
16	2	1	x
17	2	2	y
18	2	2	z
19	2	3	x
20	2	9	a

10. $\gamma_{A\,;\,count(B) \rightarrow t}(R \bowtie S)$

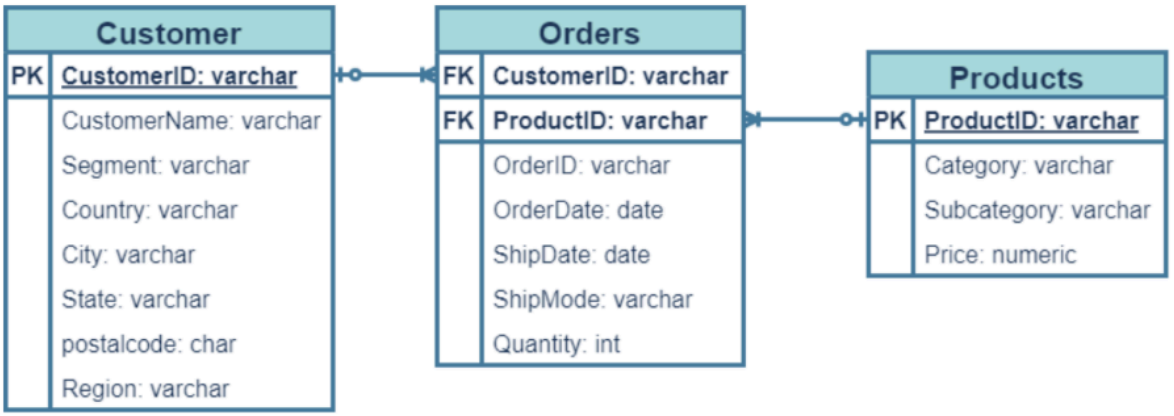
$R \bowtie S$

	A	B	C	D
1	1	x	0	3
2	3	x	0	3
3	2	y	2	1
4	2	y	3	3
5	2	y	4	2
6	2	z	null	null
7	9	a	null	null
8	null	w	3	0

	A	t
1	1	1
2	3	1
3	2	4
4	9	1
5	null	1

2. Tienda de productos en línea

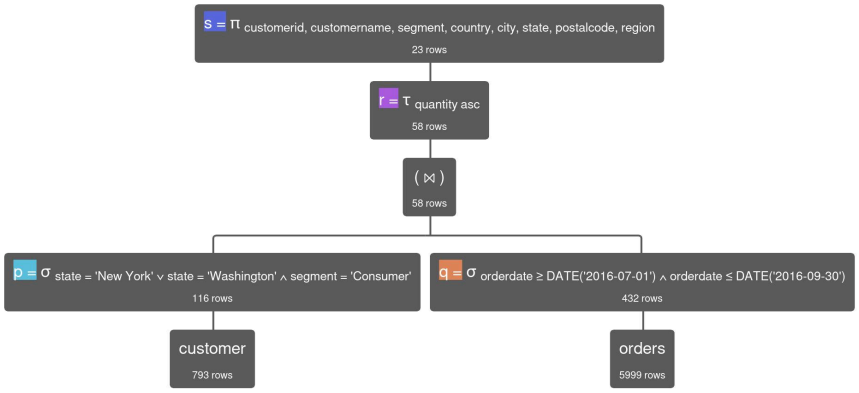
Tienes el siguiente **esquema de una base de datos** para una **tienda en línea** (ID gist: 31074567738afef8c497f6ca89335782)



Escribe una **expresión de álgebra relacional** para responder las siguientes consultas. Deberás comprobar cada una de ellas en la calculadora **Relax** y agregar **para cada inciso** la **expresión** en **álgebra relacional** y una **captura de pantalla** con el **árbol de consulta** (si es visible adecuadamente) y cada una del **resultado obtenido** (no es necesario mostrar todas las tuplas):

a. Obtener toda la información de los clientes que viven en **New York** o en **Washington**, que pertenezcan al segmento **consumer** que haya solicitado una orden en el **tercer trimestre de 2026**. Mostrar la información **ordenada** por la **cantidad solicitada**.

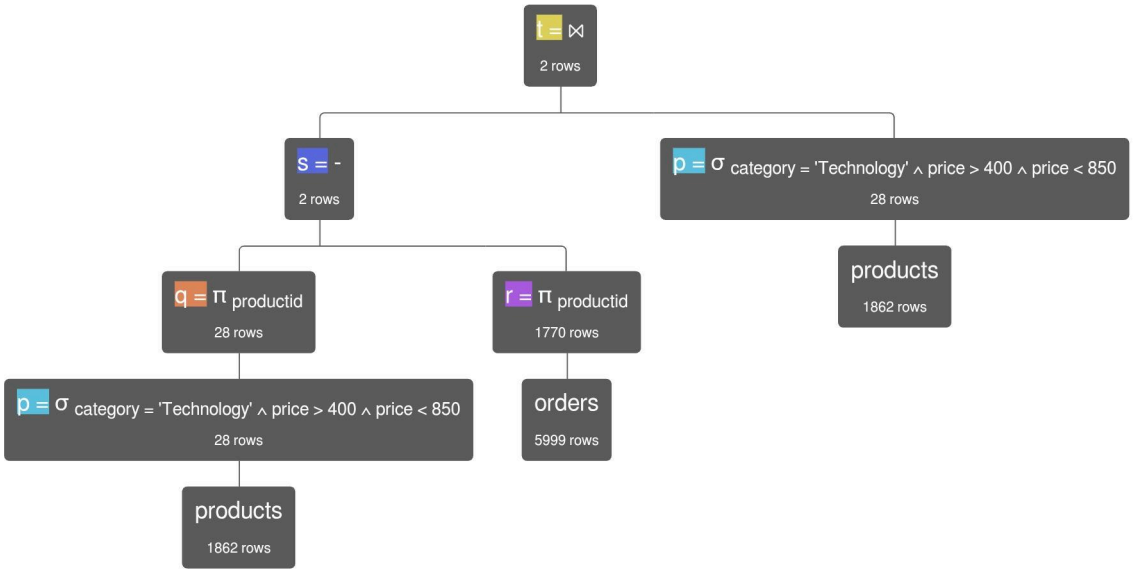
- p = σ state = 'New York'∨ state = 'Washington' ∧ segment='Consumer'(customer)
- q = σ orderdate >= date('2016-07-01') ∧ orderdate <= date('2016-09-30') orders
- r = τ quantity (p ⋈ q)
- s = π customerid, customername, segment, country, city, state, postalcode, region (r)
- s



customer.customerid	customer.customername	customer.segment	customer.country	customer.city	customer.state	customer.postalcode	customer.region
'TB-21055'	'Ted Butterfield'	'Consumer'	'United States'	'Troy'	'New York'	'12180'	'East'
'HM-14980'	'Henry MacAllister'	'Consumer'	'United States'	'New York City'	'New York'	'10009'	'East'
'PO-18850'	'Patrick OBrill'	'Consumer'	'United States'	'New York City'	'New York'	'10024'	'East'
'PS-18760'	'Pamela Stobb'	'Consumer'	'United States'	'Seattle'	'Washington'	'98105'	'West'
'CC-12370'	'Christopher Conant'	'Consumer'	'United States'	'New York City'	'New York'	'10035'	'East'
'DK-13225'	'Dean Katz'	'Corporate'	'United States'	'New York City'	'New York'	'10009'	'East'
'SE-20110'	'Sanjit Engle'	'Consumer'	'United States'	'Oceanside'	'New York'	'11572'	'East'
'MO-17500'	'Mary ORourke'	'Consumer'	'United States'	'New York City'	'New York'	'10024'	'East'
'VG-21805'	'Vivek Grady'	'Corporate'	'United States'	'New York City'	'New York'	'10035'	'East'
'RH-19600'	'Rob Haberlin'	'Consumer'	'United States'	'New York City'	'New York'	'10009'	'East'
'BC-11125'	'Becky Castell'	'Home Office'	'United States'	'New York City'	'New York'	'10035'	'East'
'SU-20665'	'Stephanie Ulpright'	'Home Office'	'United States'	'New York City'	'New York'	'10024'	'East'
'RB-19465'	'Rick Bensley'	'Home Office'	'United States'	'New York City'	'New York'	'10009'	'East'
'CC-12430'	'Chuck Clark'	'Home Office'	'United States'	'New York City'	'New York'	'10035'	'East'
'RF-19735'	'Roland Fjeld'	'Consumer'	'United States'	'Seattle'	'Washington'	'98115'	'West'
'AP-10720'	'Anne Pryor'	'Home Office'	'United States'	'New York City'	'New York'	'10035'	'East'
'MS-17770'	'Maxwell Schwartz'	'Consumer'	'United States'	'New York City'	'New York'	'10024'	'East'
'SC-20230'	'Scot Coram'	'Corporate'	'United States'	'New York City'	'New York'	'10035'	'East'
'KH-16630'	'Ken Heidel'	'Corporate'	'United States'	'New York City'	'New York'	'10009'	'East'
'NH-18610'	'Nicole Hansen'	'Corporate'	'United States'	'New York'	'New York'	'10035'	'East'
'CB-12415'	'Christy Brittain'	'Consumer'	'United States'	'New York City'	'New York'	'10035'	'East'
'RA-19285'	'Ralph Arnett'	'Consumer'	'United States'	'New York City'	'New York'	'10024'	'East'
'JJ-15445'	'Jennifer Jackson'	'Consumer'	'United States'	'Rochester'	'New York'	'14609'	'East'

b. Obtener una relación de los productos que pertenecen a la categoría **Technology** con **precio mayor** de **$400** y **menor** de **$850**, pero que **no hayan sido solicitadas** en ninguna orden.

- p = σ category = 'Technology' ∧ price > 400 ∧ price < 850 (products)
- q = π productid (p)
- r = π productid (orders)
- s = q - r
- t = s ⋈ p
- t



products.productid	products.category	products.subcategory	products.price
'TEC-PH-10004080'	'Technology'	'Phones'	487.98
'TEC-MA-10004086'	'Technology'	'Machines'	659.98

c. Obtener el **nombre** de todos los clientes que vivan en la región **South** y hayan solicitados productos de las subcategorías **Bookcases** o **Chairs** o **Appliances** o **Copiers**. El pedido debió de solicitarse el **primer** o **tercer trimestre** de **2015** y el modo de envío debe ser **Standard Class**.

Consulta:

```
clientesCandidatos = π customerid, customername (σ region = 'South' (customer))

clientes_ordenes = clientesCandidatos ⋈ orders

clientes_ordenes_filtradas = σ shipmode = 'Standard Class'

          AND ((orderdate >= date('2015-01-01') AND orderdate <= date('2015-03-31'))

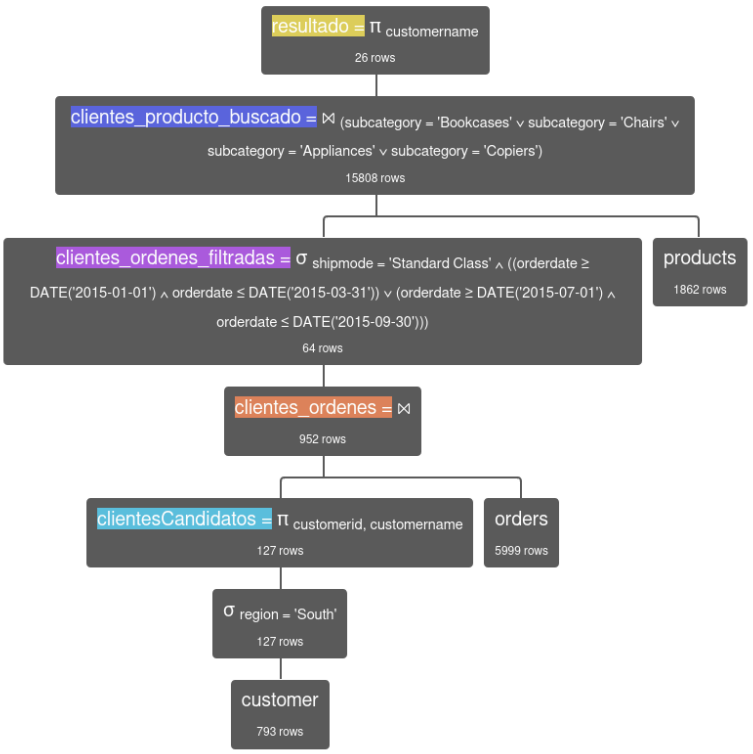
         OR (orderdate >= date('2015-07-01') AND orderdate <= date('2015-09-30'))) (clientes_ordenes)


clientes_producto_buscado = clientes_ordenes_filtradas ⋈ (

    subcategory = 'Bookcases'
```

```
    OR subcategory = 'Chairs'
    OR subcategory = 'Appliances'
    OR subcategory = 'Copiers') (products)
resultado = π customername (clientes_producto_buscado)
resultado
```

Árbol de consulta:



Resultados:

customer.customername
'Erin Smith'
'Julie Creighton'
'Paul Gonzalez'
'Philip Fox'
'Victoria Wilson'
'Sung Pak'
'Adam Bellavance'
'Mike Vittorini'
'Sarah Foster'
'Bruce Degenhardt'

customer.customername
'Sylvia Foulston'
'Cyma Kinney'
'Bryan Spruell'
'Ann Steele'
'Kalyca Meade'
'Noel Staavos'
'Cathy Armstrong'
'Corey-Lock'
'Todd Sumrall'
'Ivan Liston'

customer.customername
'Tamara Manning'
'Eleni McCrary'
'Nancy Lomonaco'
'Daniel Raglin'
'Anna Häberlin'
'Sarah Brown'

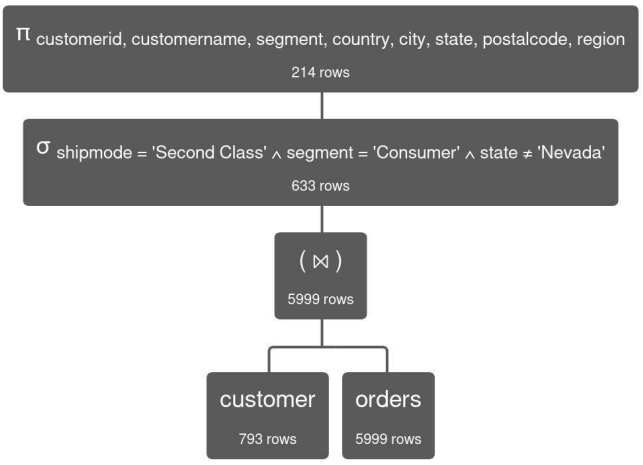
d. Toda la información de los clientes del segmento **Consumer** que realizaron una orden con modo de envío **Second Class** y que no viven en **Nevada**.

1. Primero obtenemos la tabla que contiene a todas las órdenes con **orders**.
2. Después pide como requisito que su orden de envío sea de **segunda clase**, filtramos la columna con el operador sigma.
3. Luego mezclamos las tablas de Customer con la tabla filtrada A través de un operador ⋈ join, él join junta los resultado de ambas tablas.
4. Preguntamos por el estado de los clientes que no viven en nevada.
5. Ya que al final tenemos la unión de clientes y pedidos, filtramos de tal forma que solo quedemos con atributos de clientes con el operador π.

Consulta:

π customerid, customername, segment, country, city, state, postalcode, region (σ shipmode = 'Second Class' ∧ segment = 'Consumer' ∧ state != 'Nevada' (customer ⋈ orders))

Arbol de Consulta:



Resultados:



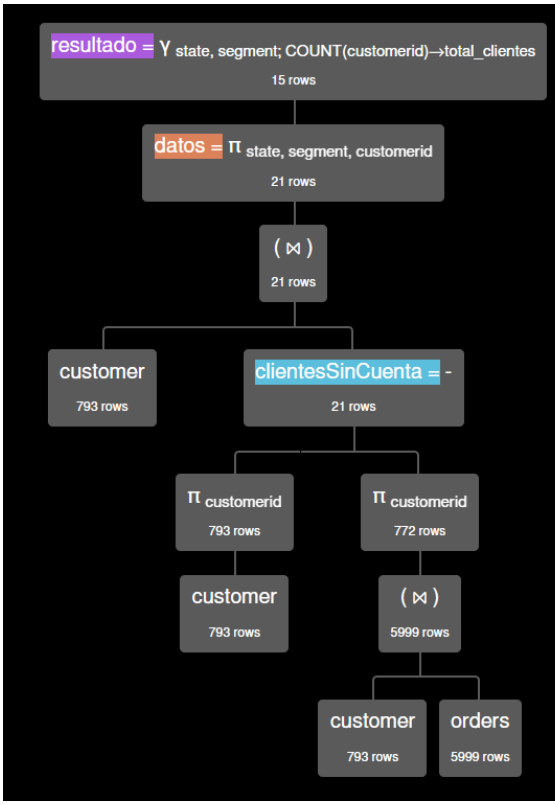
e. Obtener el **estado**, **segmento** y el **total de clientes** que no hayan solicitado **ninguna orden**.

clientesSinCuenta = (π customerid (customer)) - π customerid (customer ⋈ orders)

datos = π state,segment,customerid (customer ⋈ clientesSinCuenta)

resultado = γ state, segment; count(customerid) -> total_clientes (datos)

resultado



customer.state	customer.segment	total_clientes
'California'	'Consumer'	3
'Illinois'	'Home Office'	1
'Ohio'	'Corporate'	1
'New York'	'Corporate'	3
'Texas'	'Home Office'	1
'Delaware'	'Corporate'	2
'Tennessee'	'Corporate'	1
'California'	'Corporate'	1
'Pennsylvania'	'Corporate'	1
'Minnesota'	'Home Office'	1

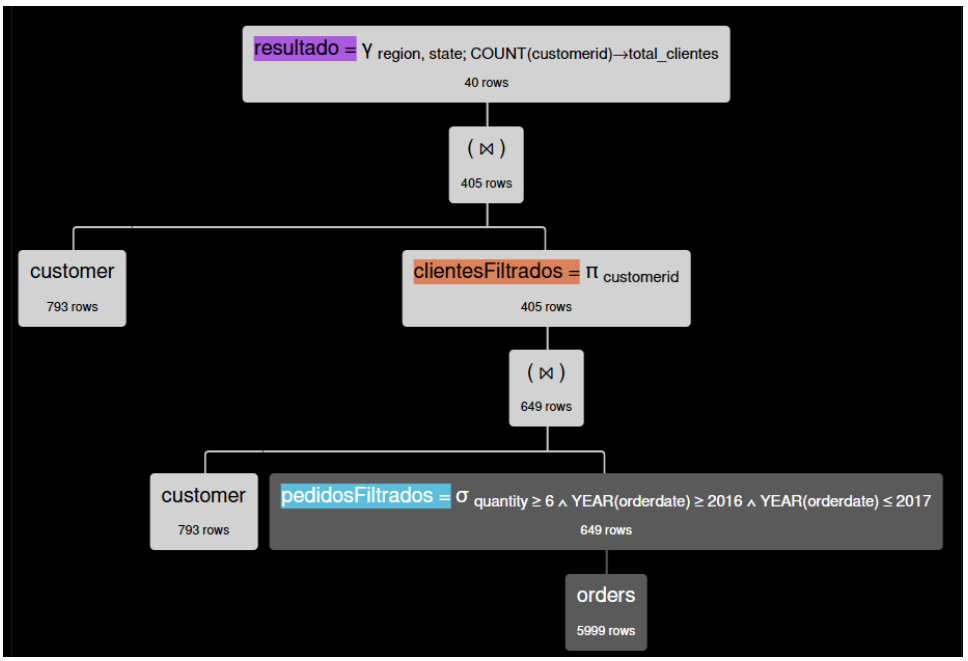
f. Una lista que muestre la **región**, el **estado** y el **total de clientes** que se tienen, considerando que los clientes deben haber realizado órdenes con **al menos 6 productos** durante **2016** o **2017**. **Ordenar** la información por **región** y **estado**.

pedidosFiltrados = σ quantity ≥ 6 ∧ year(orderdate) ≥ 2016 ∧ year(orderdate) ≤ 2017 (orders)

clientesFiltrados = π customerid (customer ⋈ pedidosFiltrados)

resultado = γ region, state; count(customerid) -> total_clientes (customer ⋈ clientesFiltrados)

resultado



customer.region	customer.state	total_clientes
'West'	'California'	82
'South'	'North Carolina'	15
'Central'	'Wisconsin'	5
'Central'	'Nebraska'	3
'West'	'Utah'	3
'Central'	'Texas'	35
'Central'	'Illinois'	21
'South'	'Florida'	15
'Central'	'Michigan'	9
'East'	'Delaware'	3

g. Obtener el **modo de envío** y **categoría** que **más productos** ha vendido.

Primero hacemos una agrupación, calculando la suma de la cantidad vendida de cada producto con el modelo de envío y la categoria (se obtienen con el natural join entre orders y products) y guardamos el total de cada producto vendido en una nueva columna llamada total (la nombramos como A para ocuparlo después).

A = γ shipmode,category;SUM(quantity)→ total(orders⋈products)

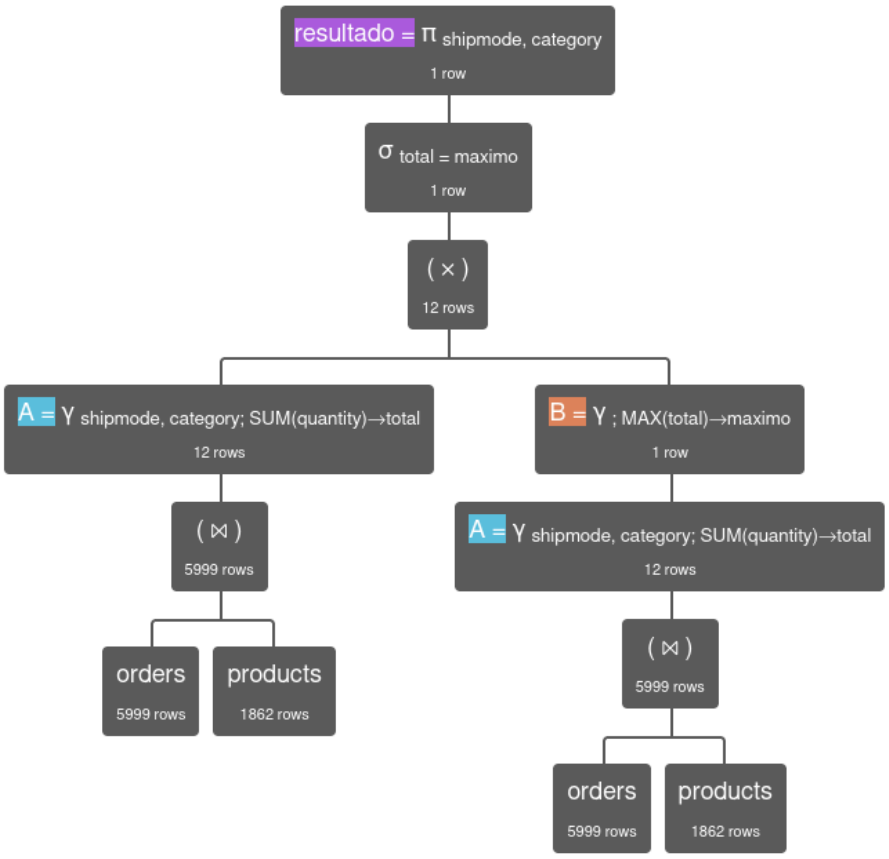
Después sacamos el máximo de todos los totales obtenidos en la instrucción anterior que nombramos A, a través de una agrupación, guardamos este resultado en una nueva columna que nombramos maximo (la relación la nombramos como B para ocuparla después).

B = γ MAX(total)→ maximo(A)

Para obtener lo deseado, hacemos una proyección que obtiene el shipmode y category únicamente del producto con la mayor cantidad de ventas (lo obtenemos comparando el total de cada producto con el maximo y si son iguales lo devuelve), por eso hacemos una selección que filtra el máximo del producto cartesiano entre A y B, y mostramos el resultado.

resultado = π shipmode,category(σ total = maximo (A×B))

resultado



orders.shipmode	products.category
'Standard Class'	'Office Supplies'

h. Una lista con la **venta promedio**, **venta total**, **mayor venta**,**menor venta** y **total de órdenes**, por **región**, **estado** y **ciudad**. La **venta promedio** debe estar entre **$900** y **$1,500**.

ProductoPedido = π region, state, city, orderid, quantity * price → importe (customer ⋈ orders ⋈ products)

Ordenes = γ region, state, city, orderid; sum(importe) → totalorden (ProductoPedido)

VentaRegional = γ region, state, city; avg(totalorden) → ventapromedio, sum(totalorden) → ventatotal, max(totalorden) → mayorventa, min(totalorden) → menorventa, count(orderid) → totalordenes (Ordenes)

σ ventapromedio >= 900 ∧ ventapromedio <= 1500 (VentaRegional)

1.Primero realizamos la unión de las tablas (customer ⋈ orders ⋈ products) a través del operador join, El orden importa porque orders es la tabla puente: comparte customerid con customer y productid con products, Luego creamos una columna llamada importe el cual registrará las ganancias que se hayan hecho de un producto, esto se calcula a partir de la cantidad de producto por su precio, dicha columna nueva se le llamara importe.

2.Después se tiene que guardar el cómo se hará el registro de los productos vendidos a partir de la nueva tabla creada llamada, ProductoPedido, para ello necesitamos los atributos que definirán el cómo serán ordenados como **región**, **estado** y **ciudad**. Además, necesitaremos acceso a los id de productos para estar sumando y contando las órdenes.

3.Una vez que los productos registrados, se necesita llevar registro de la agrupación de las órdenes, las órdenes al igual que los productos necesitan **región**, **estado** y **ciudad**, además de la id de las órdenes para tener conteo sobre ellas, la suma de todos los importes da como resultado el precio que se realizó de una orden, el cual será un atributo nuevo de tabla sacado a partir de los productos Pedidos.

4.Para obtener la ventas regionales, necesitamos agrupar con el operador gamma de acuerdo a los criterios de **región**, **estado** y **ciudad,** dicho de otra forma una vez agrupamos los pedidos, calculamos todos los resultados incluyendo los datos necesarios para realizarlos y guardandolos en atributos nuevos de la tabla.

avg(totalorden) → ventapromedio (el promedio total de las órdenes por zona)

sum(totalorden) → ventatotal (la venta total de las órdenes por zona)

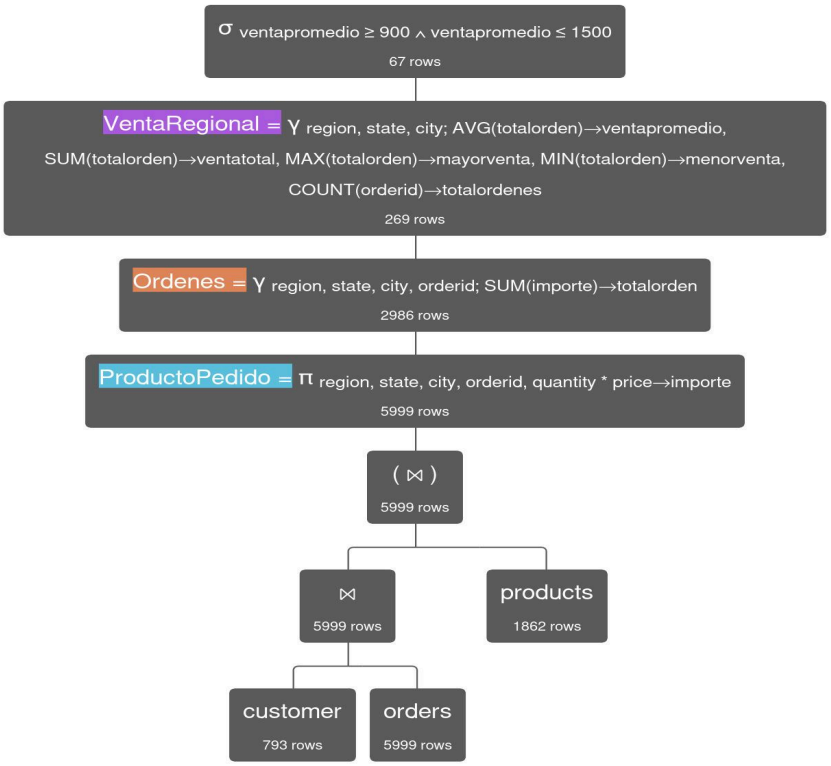
max(totalorden) → mayorventa (la orden que obtuvo mayor venta por zona)

min(totalorden) → menorventa (la orden que obtuvo menor venta por zona)

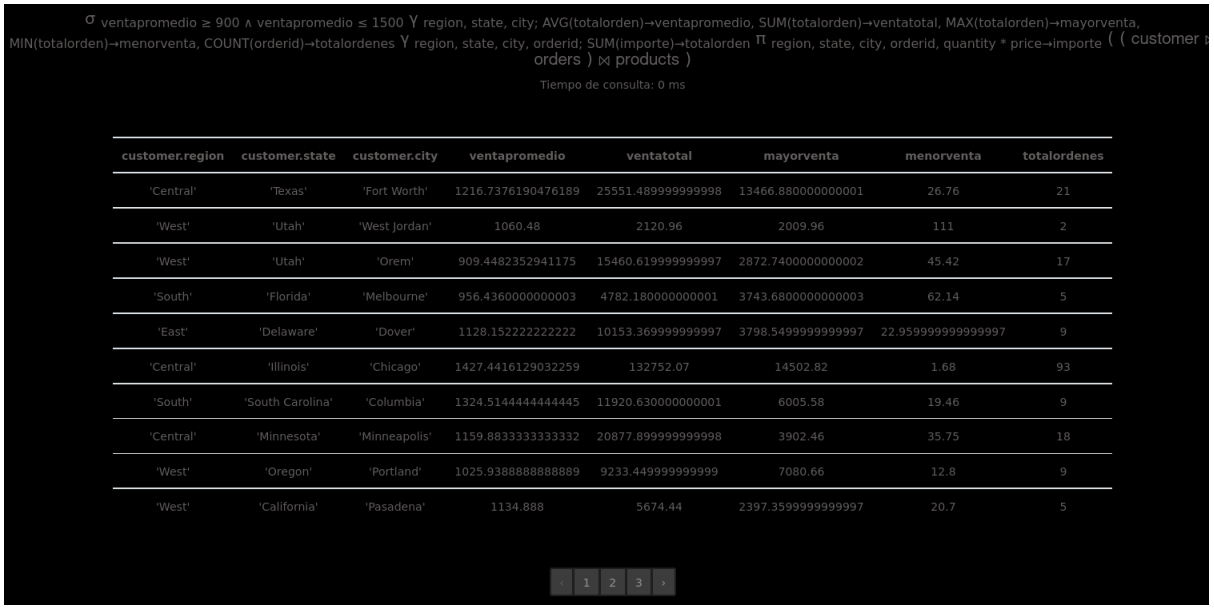
count(orderid) → totalordenes (Ordenes) (el total de órdenes realizadas por zona)

5.Finalmente pide que el promedio de venta ronde entre los **$900** y **$1,500**, eso se consigue a través del operador sigma que filtra las ventas promedio sacadas a partir de la venta regional.

Arbol de consulta:



Resultado:



customer.region	customer.state	customer.city	ventapromedio	ventatotal	mayorventa	menorventa	totalordenes
'Central'	'Texas'	'Fort Worth'	1216.7376190476189	25551.489999999998	13466.880000000001	26.76	21
'West'	'Utah'	'West Jordan'	1060.48	2120.96	2009.96	111	2
'West'	'Utah'	'Orem'	909.4482352941175	15460.619999999997	2872.7400000000002	45.42	17
'South'	'Florida'	'Melbourne'	956.4360000000003	4782.180000000001	3743.6800000000003	62.14	5
'East'	'Delaware'	'Dover'	1128.152222222222	10153.369999999997	3798.5499999999997	22.959999999999997	9
'Central'	'Illinois'	'Chicago'	1427.4416129032259	132752.07	14502.82	1.68	93
'South'	'South Carolina'	'Columbia'	1324.5144444444445	11920.630000000001	6005.58	19.46	9
'Central'	'Minnesota'	'Minneapolis'	1159.8833333333332	20877.899999999998	3902.46	35.75	18
'West'	'Oregon'	'Portland'	1025.9388888888889	9233.449999999999	7080.66	12.8	9
'West'	'California'	'Pasadena'	1134.888	5674.44	2397.3599999999997	20.7	5

i. El estado que ha realizado la **menor cantidad de órdenes**. Se debe mostrar también el **total de órdenes** que haya entregado.

Primero contamos la cantidad de pedidos totales que tiene cada producto, a través de una agrupación, guardamos cada cantidad en una nueva columna llamada totalpedidos (la nombramos como A para ocuparla después), tomamos solamente como órdenes entregadas a las que tienen shipdate distinta a NULL (ya que no hay algún estatus que las marque como entregados los pedidos) para mostrar el total de órdenes entregadas.

A = γ state;COUNT(orderid) → totalpedidos(customer ⋈ σ shipdate ≠ NULL (orders))

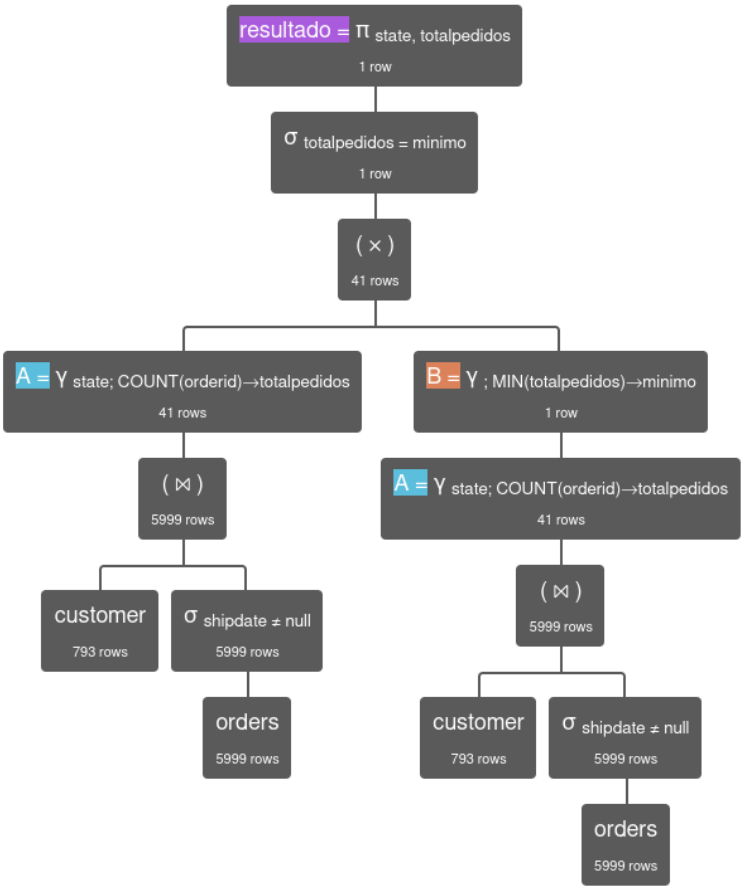
Después a partir de los resultados anteriores, buscamos el del valor mínimo con una agrupación y lo guardamos en una nueva columna llamada minimo (la relación la nombramos como B para usarla después)

B = γ MIN(totalpedidos) → minimo(A)

Para obtener el resultado esperado hacemos una proyección que obtiene el state y totalpedidos del producto con la menor cantidad deión que obtiene el state y totalpedidos del producto con la menor cantidad de pedidos, y finalmente lo mostramos.

resultado = π state, totalpedidos (σ totalpedidos = minimo (A×B))

resultado



customer.state	totalpedidos
'District of Columbia'	4

j. La información del cliente que más órdenes haya efectuado. Mostrar el número de órdenes que ha realizado.

Consulta:

```
-- 1. Tomamos las id's de las ordenes y de los clientes que las hicieron.

ids_orders = π orderid,customerid (orders)

clientes_ordenes = customer ⋈ ids_orders

-- 2. Creamos una tabla con los identificadores de los clientes y el número de ordenes que han realizado.

clientes_num_ordenes = γ customerid,customername,segment,country,city,state,postalcode,region; count(orderid) -> total_ordenes (clientes_ordenes)
```

```
-- 3. Creamos una tabla que coontenga el número máximo de pedidos que ha realizado cualquier cliente.

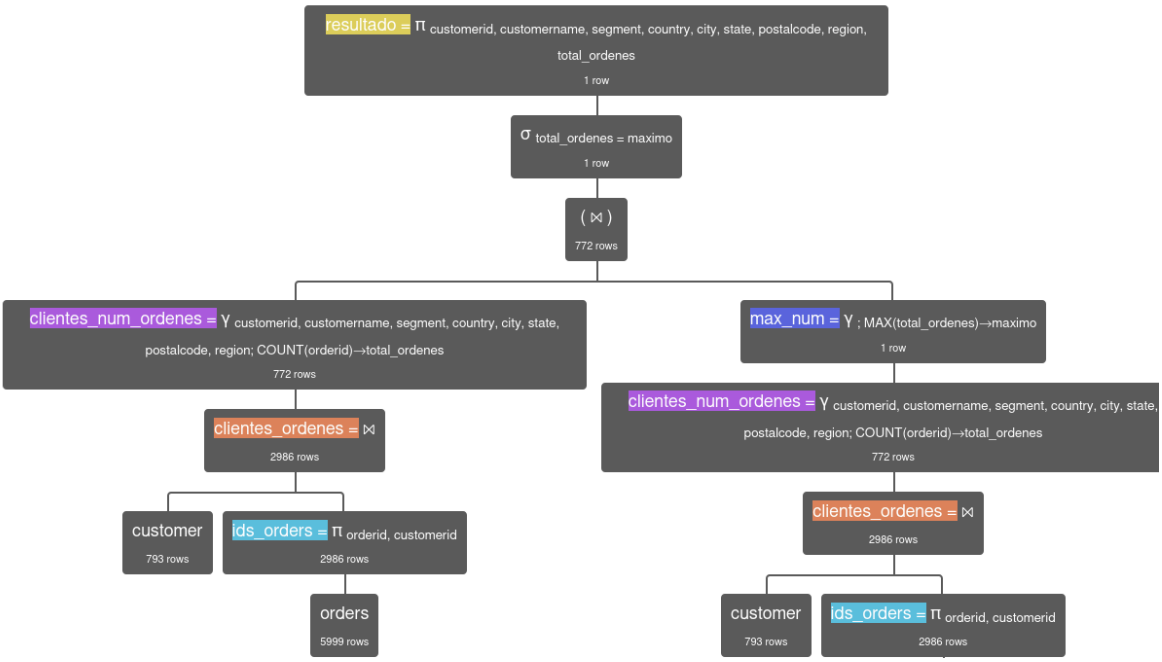
max_num =  γ max(total_ordenes) -> maximo (clientes_num_ordenes)

-- 4 Combinamos los clientes y sus números de ordenes con el número maximo de ordenes. Después, tomamos a los clientes con sus números de ordenes y el maximo tales que: números de ordenes = maximo. Usamos una proyección para eliminar estos atributos y quedarnos solo con la información del cliente.

resultado = π customerid, customername, segment, country, city, state, postalcode, region , total_ordenes (σ total_ordenes = maximo (clientes_num_ordenes ⋈ max_num))

resultado
```

Árbol de consulta:



Resultado:

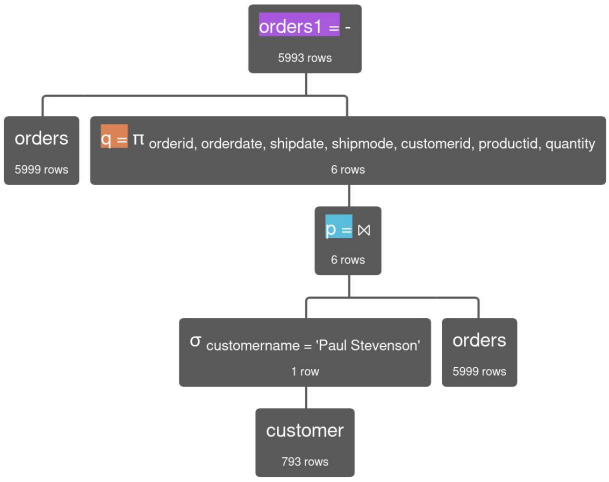
customer.customerid	customer.customername	customer.segment	customer.country	customer.city	customer.state	customer.postalcode	customer.region	total_ordenes
'JE-15745'	'Joel Eaton'	'Consumer'	'United States'	'Jackson'	'Michigan'	'49201'	'Central'	13

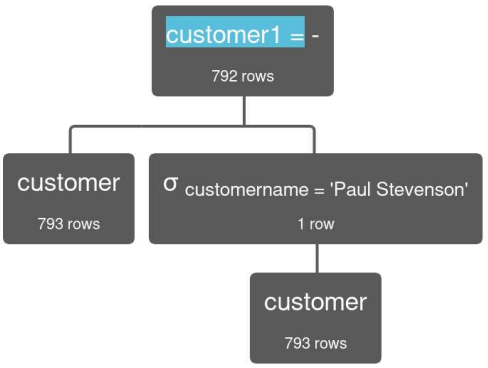
‹ 1 ›

Operaciones de mantenimiento de datos: borrado, inserción y actualización

a. Borrar toda la información del cliente Paul Stevenson.

- p = σ customername = 'Paul Stevenson' (customer) ⋈ orders
- q = π orderid, orderdate, shipdate, shipmode, customerid, productid, quantity (p)
- orders1 = orders - q
- -- orders1
- customer1 = customer - σ customername = 'Paul Stevenson' (customer)
- customer1





customer.customerid	customer.customername	customer.segment	customer.country	customer.city	customer.state	customer.postalcode	customer.region
'CG-12520'	'Claire Gute'	'Consumer'	'United States'	'Henderson'	'Kentucky'	'42420'	'South'
'DV-13045'	'Darrin Van Huff'	'Corporate'	'United States'	'Los Angeles'	'California'	'90036'	'West'
'SO-20335'	'Sean ODonnell'	'Consumer'	'United States'	'Fort Lauderdale'	'Florida'	'33311'	'South'
'BH-11710'	'Brosina Hoffman'	'Consumer'	'United States'	'Los Angeles'	'California'	'90032'	'West'
'AA-10480'	'Andrew Allen'	'Consumer'	'United States'	'Concord'	'North Carolina'	'28027'	'South'
'IM-15070'	'Irene Maddox'	'Consumer'	'United States'	'Seattle'	'Washington'	'98103'	'West'
'HP-14815'	'Harold Pawlan'	'Home Office'	'United States'	'Fort Worth'	'Texas'	'76106'	'Central'
'PK-19075'	'Pete Kriz'	'Consumer'	'United States'	'Madison'	'Wisconsin'	'53711'	'Central'
'AG-10270'	'Alejandro Grove'	'Consumer'	'United States'	'West Jordan'	'Utah'	'84084'	'West'
'ZD-21925'	'Zuschuss Donatelli'	'Consumer'	'United States'	'San Francisco'	'California'	'94109'	'West'

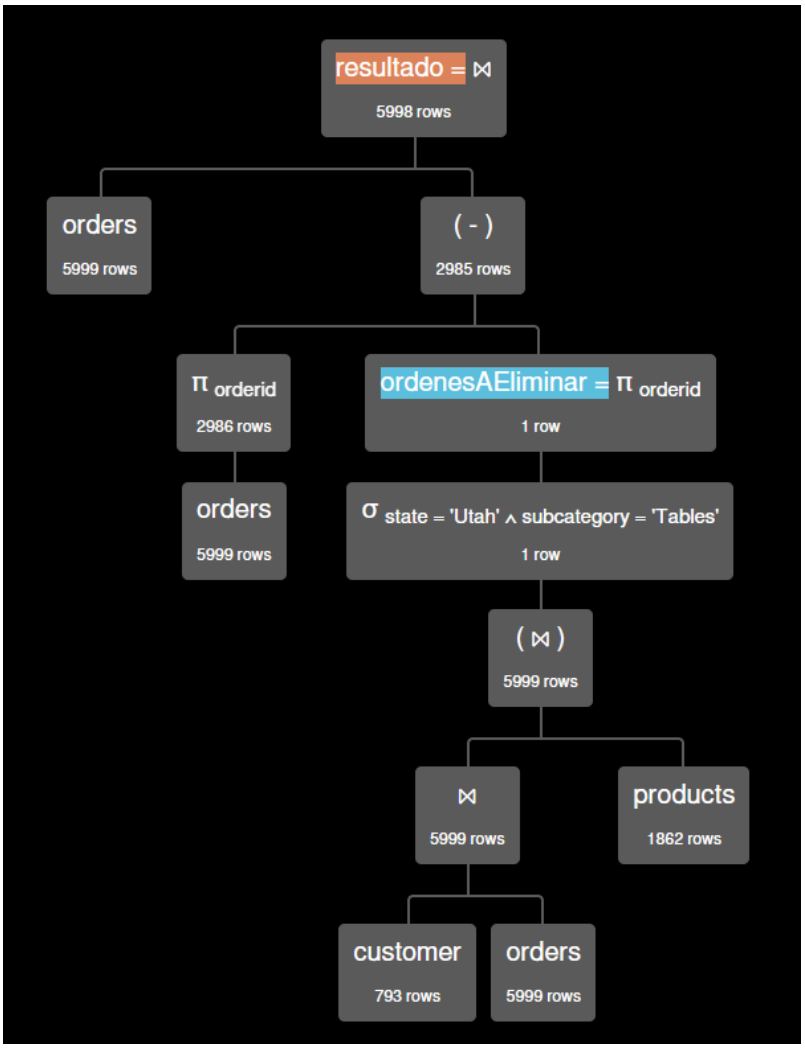
‹ 1 2 3 ›

b. Borrar todas las órdenes de la ciudad **Utah** que tengan artículos de la subcategoría **Tables**.

ordenesAEliminar = π orderid (σ state = 'Utah' ∧ subcategory = 'Tables' (customer ⋈ orders ⋈ products))

resultado = orders ⋈ (π orderid (orders) - ordenesAEliminar)

resultado



orders.orderid	orders.orderdate	orders.shipdate	orders.shipmode	orders.customerid	orders.productid	orders.quantity
'CA-2016-152156'	2016-11-08	2016-11-11	'Second Class'	'CG-12520'	'FUR-BO-10001798'	2
'CA-2016-152156'	2016-11-08	2016-11-11	'Second Class'	'CG-12520'	'FUR-CH-10000454'	3
'CA-2016-138688'	2016-06-12	2016-06-16	'Second Class'	'DV-13045'	'OFF-LA-10000240'	2
'US-2015-108966'	2015-10-11	2015-10-18	'Standard Class'	'SO-20335'	'FUR-TA-10000577'	5
'US-2015-108966'	2015-10-11	2015-10-18	'Standard Class'	'SO-20335'	'OFF-ST-10000760'	2
'CA-2014-115812'	2014-06-09	2014-06-14	'Standard Class'	'BH-11710'	'FUR-FU-10001487'	7
'CA-2014-115812'	2014-06-09	2014-06-14	'Standard Class'	'BH-11710'	'OFF-AR-10002833'	4
'CA-2014-115812'	2014-06-09	2014-06-14	'Standard Class'	'BH-11710'	'TEC-PH-10002275'	6
'CA-2014-115812'	2014-06-09	2014-06-14	'Standard Class'	'BH-11710'	'OFF-BI-10003910'	3
'CA-2014-115812'	2014-06-09	2014-06-14	'Standard Class'	'BH-11710'	'OFF-AP-10002892'	5

c. La clienta **Lena Cacioppo** compró un producto de cada subcategoría de **Furniture**. Deberás elegir los productos que desees e indicar como parte de esta consulta, la información que se agregara en cada caso.

Consulta:

```
-- 1. Conseguimos el id de Lena

Lena = σ customername = 'Lena Cacioppo' (customer)

-- 2. Conseguimos el id de un producto de cada categoría especifica. Se agrega el id de lena por conveniencia.

id_por_subcategoria = γ subcategory; min(productid) -> productid (products)


-- 3. Cruzamos a Lena con los 17 productos para tener (customerid, productid)

Lena_Productos = π customerid, productid (Lena × id_por_subcategoria)


-- 4. Relación en línea con la lista de órdenes

Detalles = {

   orderid:string, orderdate:date, shipdate:date, shipmode:string, quantity:number

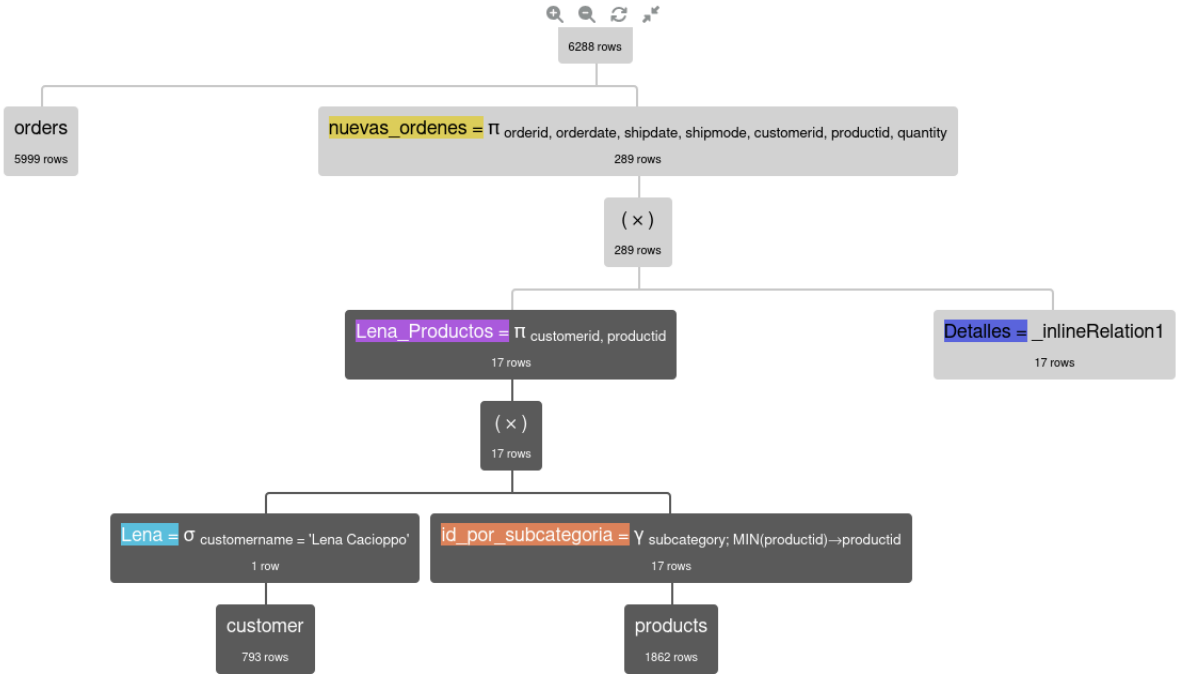
  US-2026-100001, 2026-10-01, 2026-10-04, 'Standard Class', 3
```

```
  US-2026-100002, 2026-10-02, 2026-10-06, 'Second Class', 5
  US-2026-100003, 2026-10-03, 2026-10-03, 'Same Day', 2
  US-2026-100004, 2026-10-04, 2026-10-09, 'Standard Class', 7
  US-2026-100005, 2026-10-05, 2026-10-07, 'First Class', 1
  US-2026-100006, 2026-10-06, 2026-10-10, 'Second Class', 4
  US-2026-100007, 2026-10-07, 2026-10-12, 'Standard Class', 9
  US-2026-100008, 2026-10-08, 2026-10-10, 'First Class', 2
  US-2026-100009, 2026-10-09, 2026-10-09, 'Same Day', 6
  US-2026-100010, 2026-10-10, 2026-10-13, 'Second Class', 3
  US-2026-100011, 2026-10-11, 2026-10-13, 'First Class', 8
  US-2026-100012, 2026-10-12, 2026-10-16, 'Standard Class', 5
  US-2026-100013, 2026-10-13, 2026-10-18, 'Standard Class', 1
  US-2026-100014, 2026-10-14, 2026-10-17, 'Second Class', 4
  US-2026-100015, 2026-10-15, 2026-10-15, 'Same Day', 2
  US-2026-100016, 2026-10-16, 2026-10-18, 'First Class', 6
  US-2026-100017, 2026-10-17, 2026-10-21, 'Standard Class', 3
}


-- 5. Proyectamos las columnas en el orden EXACTO que exige 'orders'
nuevas_ordenes = π orderid, orderdate, shipdate, shipmode, customerid, productid, quantity (Lena_Productos × Detalles)


-- 6. Unión final
orders ∪ nuevas_ordenes -- Se agregan las nuevas ordenes.
```

Árbol de consulta:



Resultados:

orders.orderid	orders.orderdate	orders.shipdate	orders.shipmode	orders.customerid	orders.productid	orders.quantity
'CA-2016-152156'	2016-11-08	2016-11-11	'Second Class'	'CG-12520'	'FUR-BO-10001798'	2
'CA-2016-152156'	2016-11-08	2016-11-11	'Second Class'	'CG-12520'	'FUR-CH-10000454'	3
'CA-2016-138688'	2016-06-12	2016-06-16	'Second Class'	'DV-13045'	'OFF-LA-10000240'	2
'US-2015-108966'	2015-10-11	2015-10-18	'Standard Class'	'SO-20335'	'FUR-TA-10000577'	5
'US-2015-108966'	2015-10-11	2015-10-18	'Standard Class'	'SO-20335'	'OFF-ST-10000760'	2
'CA-2014-115812'	2014-06-09	2014-06-14	'Standard Class'	'BH-11710'	'FUR-FU-10001487'	7
'CA-2014-115812'	2014-06-09	2014-06-14	'Standard Class'	'BH-11710'	'OFF-AR-10002833'	4
'CA-2014-115812'	2014-06-09	2014-06-14	'Standard Class'	'BH-11710'	'TEC-PH-10002275'	6
'CA-2014-115812'	2014-06-09	2014-06-14	'Standard Class'	'BH-11710'	'OFF-BI-10003910'	3
'CA-2014-115812'	2014-06-09	2014-06-14	'Standard Class'	'BH-11710'	'OFF-AP-10002892'	5

‹ 1 2 3 ›

d. Aumentar los precios de productos de la subcategoría **Phones** en un **8%.**

Primero calculamos el nuevo precio de los productos que son de subcategoría ‘Phone’, aumentamos el precio en un 8%, esto lo calculamos multiplicando el precio original por 1.08 (nombramos la relación como A para ocuparla después).

A = π productid, category, subcategory, price ← price*(1.08) (σ subcategory = 'Phones'(products))

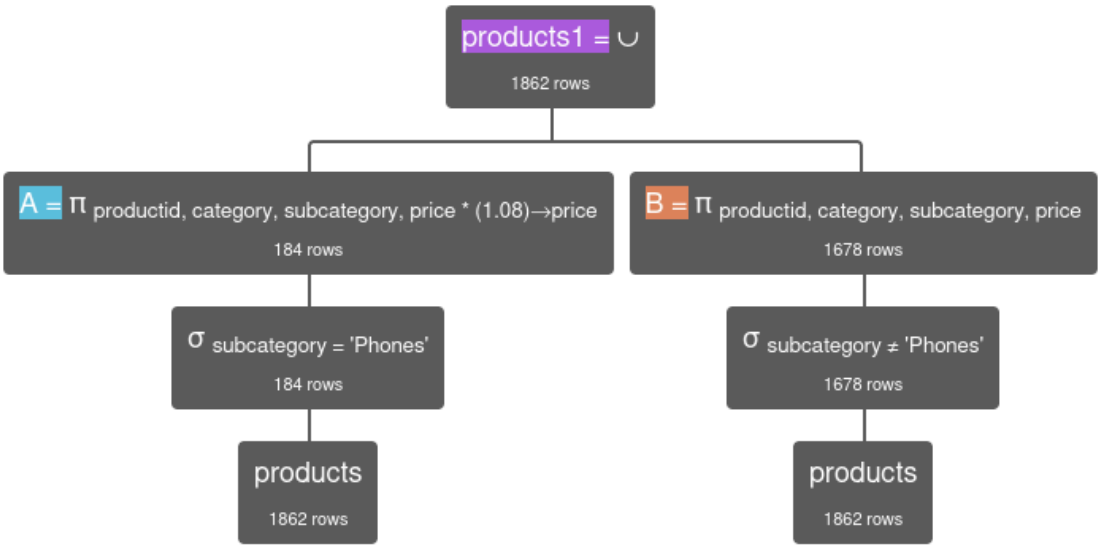
Después nombramos a B todas las demás filas que no fueron modificadas.

B = π productid, category, subcategory, price (σ subcategory ≠ 'Phones'(products))

Hacemos la unión de A con B y mostramos los resultados.

products1 = A ∪ B

products1



products.productid	products.category	products.subcategory	price
'TEC-PH-10002275'	'Technology'	'Phones'	979.7220000000001
'TEC-PH-10002033'	'Technology'	'Phones'	984.3336
'TEC-PH-10001949'	'Technology'	'Phones'	230.5584
'TEC-PH-10004977'	'Technology'	'Phones'	31.8276
'TEC-PH-10000486'	'Technology'	'Phones'	229.02480000000003
'TEC-PH-10004093'	'Technology'	'Phones'	400.8636
'TEC-PH-10003988'	'Technology'	'Phones'	48.6
'TEC-PH-10002447'	'Technology'	'Phones'	16.480800000000002
'TEC-PH-10002726'	'Technology'	'Phones'	230.16960000000003
'TEC-PH-10002844'	'Technology'	'Phones'	110.5488

‹ 1 2 3 ›

e. Disminuir **8%** los precios de los productos de la categoría **Furniture** cuyo precio sea de **$600** a **$900**. Aumentar en un **5%** los precios de los productos de la categoría **Technology** y subcategoría **Machines**.

FurnMedio = σ category = 'Furniture' ∧ price >= 600 ∧ price <= 900 (products)

TechMachines = σ category = 'Technology' ∧ subcategory = 'Machines' (products)

SinCambio = products \ FurnMedio \ TechMachines

FurnNuevo = π productid, category, subcategory, price * 0.92 -> price (FurnMedio)

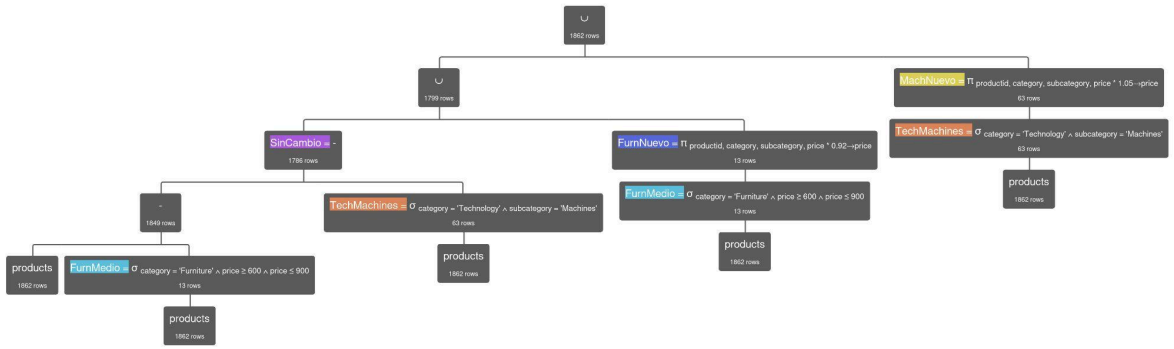
MachNuevo = π productid, category, subcategory, price * 1.05 -> price (TechMachines)

SinCambio ∪ FurnNuevo ∪ MachNuevo

1. Primero nos pide disminuir el precio de los muebles que entran en la categoría Furniture, cuyo precio sea de entre 600$ y 900$, esto es realizado por la primera línea a través del operador sigma que solo se queda con las filas que cumplan todo lo anteriormente dicho, lo cual lo guardaremos en una tabla nueva para tenerlo registrado.
2. Después definimos la segunda tabla que engloba los elementos de de los productos de la categoría **Technology** y con subcategoría **Machines,(**la subcategoría necesita que se saque de producto para filtrar**)**, todo lo anterior una vez más filtrado por el operador sigma.

3. De la tabla principal guardamos únicamente las columnas de las que no realizamos ningún cambio.
4. Creamos una nueva tabla que guarde los cambios registrados para los muebles que les disminuya el precio un 8% a partir de su precio original.
5. Creamos una nueva tabla que guarde los cambios realizados con respecto a los precios de los productos tecnológicos con subcategoría máquina, donde se les aumente el precio un 5% con respecto a su precio original.
6. Finalmente volvemos a juntar los datos no cambiados con las tablas que realizan los nuevos cambios para tener la actualización requerida en todos los productos proporcionados.

Árbol de consulta:



Resultado:



products.productid	products.category	products.subcategory	products.price
'FUR-BO-10001798'	'Furniture'	'Bookcases'	261.96
'OFF-LA-10000240'	'Office Supplies'	'Labels'	14.62
'FUR-TA-10000577'	'Furniture'	'Tables'	957.58
'OFF-ST-10000760'	'Office Supplies'	'Storage'	22.37
'FUR-FU-10001487'	'Furniture'	'Furnishings'	48.86
'OFF-AR-10002833'	'Office Supplies'	'Art'	7.28
'TEC-PH-10002275'	'Technology'	'Phones'	907.15
'OFF-BI-10003910'	'Office Supplies'	'Binders'	18.5
'OFF-AP-10002892'	'Office Supplies'	'Appliances'	114.9
'FUR-TA-10001539'	'Furniture'	'Tables'	1706.18